

Sincronización de procesos

Introducción

- Existe la necesidad de comunicar los procesos entre sí.
- El acceso concurrente a datos compartidos puede dar pie a inconsistencia de datos
- Mantener la consistencia de los datos requiere mecanismos para asegurar el orden de ejecución de los procesos que los comparten
- En la electrónica se utiliza una señal de reloj para sincronizar eventos.
- Posible modos de comunicación:
 - Por memoria común: Espacio de direcciones único (ejemplo: hilos), o el sistema operativo crea explícitamente una zona de memoria común para que lo utilicen varios procesos.
 - Por mensajes: Normalmente se utiliza cuando el espacio de direcciones de dos procesos es totalmente distinto

Condición de carrera o de concurso

- En los sistemas operativos, cuando varios procesos se ejecutan a la vez con el propósito de completar una tarea se pueden dar **condiciones de carrera o de concurso**, que pudieran desembocar en un estado inesperado.
- Se da cuando el estado de un proceso es dependiente de una secuencia de eventos que se ejecutan en orden arbitrario y van a trabajar sobre un mismo recurso compartido.

Ejemplo, usando una analogía: si el recurso es “ropa” y hay 2 procesos “vestirse” y “planchar”, aunque ambos se pueden ejecutar de manera independiente, cuando ambos necesiten usar el mismo recurso tiene que existir un orden sincrónico al usarse. Primero el individuo “plancha la ropa” y luego “se viste”.

Secciones críticas

¿Cómo se evitan las condiciones de concurso?

- La clave está en prevenir el problema donde se comparta algo: memoria, archivos, puertos...
- Se debe determinar de alguna manera **prohibir** que un proceso lea o escriba datos compartidos al mismo tiempo. Este mecanismo se denomina **exclusión mutua**.
- La parte de un proceso donde se accesa a un recurso compartido se llama **sección crítica**, y aquí es donde se debe arreglar los acontecimientos para que dos procesos no accedan a sus secciones críticas al mismo tiempo.

El Problema de la Sección Crítica

- Cada proceso posee un fragmento de código de **sección crítica**, que no debe intercalarse con las secciones críticas de los demás procesos
- En las secciones críticas de los procesos se encuentra el código que accede y/o modifica los datos compartidos
- La ejecución de las secciones críticas debe ser **mutuamente exclusiva** para evitar inconsistencia de datos
- El problema de la sección crítica consiste en diseñar un protocolo que los procesos pueden usar para conseguir la exclusión mutua de las secciones críticas.
- El protocolo consta de:
 - **Sección de ingreso**: solicita permiso para ingresar en la sección crítica
 - **Sección de egreso**: anuncia la salida de la sección crítica

Condiciones para la solución al problema de la Sección Crítica

1. **Exclusión Mutua** – Si el proceso P_i está ejecutando su sección crítica, ningún otro proceso puede estar ejecutando su sección crítica
2. **Progreso** – Si ningún proceso está ejecutando su sección crítica y existen algunos que quieren entrar en su sección crítica, sólo los procesos que no estén ejecutando su sección restante pueden participar en la decisión de qué proceso puede ingresar en su sección crítica, y esta selección no puede posponerse indefinidamente
3. **Espera limitada** - Hay un límite para el número de veces que otros procesos pueden entrar a sus secciones críticas después de que un proceso ha solicitado entrar en su sección crítica y antes de que se le otorgue la autorización para hacerlo
 - Asumimos que cada proceso se ejecuta con velocidad $\neq 0$
 - No hacemos supuestos acerca de las velocidades relativas de los N procesos

Primer intento

```
while (true){  
    while (turno != 0); //espera  
    seccion_critica();  
    turno = 1;  
    seccion_NO_critica();  
}
```

Este método se denomina **alternancia estricta** usando una **variable de cierre**

- Satisface la exclusión mutua
- No cumple la condición de progreso
- Requiere una alternancia estricta de los procesos en la ejecución de la sección crítica
- Por lo general se debe evitar, ya que desperdicia tiempo de CPU por la continua comprobación de una variable de espera.

Segundo intento

```
while (true){  
    indicador[0] = TRUE;  
    while (indicador[1]);  
    seccion_critica();  
    indicador[0] = FALSE;  
    seccion_NO_critica();  
}
```

- Satisface la exclusión mutua
- No cumple la condición de progreso
- Los dos procesos pueden quedarse bloqueados en ciclos infinitos

Solución de Dekker (1965)

```
while (true){
    indicador[0] = TRUE;
    while (indicador[1]){
        if (turno ≠ 0){
            indicador[0] = FALSE;
            while (turno ≠ 0);
            indicador[0] = TRUE;
        }
    }
    seccion_critica();
    turno = 1;
    indicador[0] = FALSE;
    seccion_NO_critica();
}
```

El matemático alemán T. Dekker fue el primero en crear una solución por medio del software el problema de la exclusión mutua que no requiere alternancia estricta, al combinar la idea de los turnos con las variables de cierre y advertencia.

- Satisface la exclusión mutua
- Cumple la condición de progreso
- Cumple el requisito de espera limitada
- El problema es que la solución planteada es muy complicada.

Solución de Peterson (1981)

- Peterson descubrió una manera más simple que Dekker para lograr la exclusión mutua.
- Asume que las instrucciones de carga y almacenamiento (LOAD y STORE) son atómicas; no pueden ser interrumpidas
- Los dos procesos comparten dos variables:
 - int **turno**
 - Boolean **indicador[2]**
- La variable **turno** indica a quién le toca entrar en la sección crítica
- Los **indicadores** se usan para indicar si un proceso está listo para entrar en la sección crítica. **indicador[i]** = TRUE implica que el proceso P_i está listo

Algoritmo para el Proceso

```
#define N 2    // Número de procesos
int turno;    // ¿a quién le toca?
bool indicador[N]; // todos inicialmente false

void entrar_region_critica(int proceso){
    int otro; // número del otro proceso
    otro = 1 - proceso; // Opuesto al proceso
    indicador[proceso] = true; //demuestre su interés
    turno = proceso;
    while (indicador[otro] && turno == proceso); //proposición nula
}

void abandonar_region_critica(int proceso){
    indicador[proceso] = false; //indicar abandono
}
```

- Satisface la exclusión mutua
- Cumple la condición de progreso
- Cumple el requisito de espera limitada

Hardware de Sincronización

- Muchos sistemas proveen soporte hardware para resolver el problema de la exclusión mutua (TSL – Test and Set Lock)
- Una solución en máquinas con un solo procesador es deshabilitar las interrupciones
 - El código que se está ejecutando no puede ser retirado de la CPU
 - No es buena solución porque el SO pierde el control temporalmente
 - En sistemas multiprocesadores no es eficiente
- Las máquinas actuales proveen instrucciones atómicas especiales
 - Atómica = no interrumpible
 - Chequeo y asignación simultánea
 - Intercambio de dos palabras de memoria

Instrucción Test & Set (TSL)

```
boolean TestAndSet (boolean *target) {  
    boolean rv = *target;  
    *target = TRUE;  
    return rv;  
}
```

Usando ensamblador ficticio:

```
entrar_region:  
;copiar la señal en el registro  
    tsl    register, flag  
; fue 0 la señal?  
    cmp    register, #0  
;dif de 0, cerrojo  
    jnz    entrar_region  
;regresar y región crítica introducida  
    ret  
  
abandonar_region:  
;almacenar 0 en la señal  
    mov    flag, #0  
    ret    ;regresar
```

Solución usando Test & Set

- Se comparte una variable booleana *lock*, inicializada a *false*.
- Es una operación que se ejecuta de forma atómica, es decir, no cabe la posibilidad de que su ejecución se interrumpa a la mitad.

```
while (true) {  
    while ( TestAndSet (&lock )); // nada  
    //  sección crítica  
  
    lock = FALSE;  
    //  sección restante  
}
```

Instrucción Swap

Permite intercambiar atómicamente el valor de dos variables en una sola operación.

```
void Swap (boolean *a, boolean *b){  
    boolean temp = *a;  
    *a = *b;  
    *b = temp;  
}
```

```
while (true) {  
    key = TRUE;  
    while ( key == TRUE)  
        Swap (&lock, &key );  
    seccion_critica();  
    lock = FALSE;  
    seccion_NO_critica();  
}
```

Solución usando **Swap**

- Se comparte una variable booleana *lock* inicializada a FALSE (recurso libre).
- Cada proceso tiene una variable local booleana *key*
- Con la operación swap atómica...
 - Se garantiza la exclusión mutua sobre el recurso compartido,
 - También el progreso
 - Pero no la condición de espera limitada ... (habría que hacer algo más para solucionarlo, pero no nos meteremos).

Semáforos

- Este era el problema cuando Dijkstra (1965) sugirió el uso de una variable entera para contar los desbloques guardados.
- Mecanismo de sincronización de procesos basado en el envío y recepción de señales.
- Herramienta de sincronización que no requiere espera activa
- Semáforo S – variable entera
- Dos operaciones estándar modifican S : $\text{wait}()$ y $\text{signal}()$
 - Llamadas originalmente por Dijkstra $P()$ proberen = probar y $V()$ verhogen = incrementar, en holandés.
- Sólo puede accederse al semáforo a través de las dos operaciones atómicas

```
wait (S) {  
    while S <= 0; // no-op  
    S--;  
}
```

```
signal (S) {  
    S++;  
}
```

Semáforo como Herramienta de Sincronización

- Semáforo de conteo – el valor entero puede variar en un dominio no acotado
- Semáforo binario – el valor entero puede variar sólo entre 0 y 1
 - También se conoce como `mutex locks`
- Se puede implementar un semáforo de conteo usando un semáforo binario
- Uso de semáforo para exclusión mutua
 - Semaphore S; // inicializado a 1
 - wait (S);
 Sección Crítica
 signal (S);

Implementación de Semáforos

- Se debe garantizar que dos procesos no ejecuten `wait ()` y `signal ()` sobre el mismo semáforo al mismo tiempo
- La operación `wait` puede implementarse con espera activa
 - Si la sección crítica es corta la espera activa también lo será
- Las aplicaciones pueden pasar mucho tiempo en secciones críticas y por tanto, no es una buena solución
 - Se desaprovecha la CPU

Implementación de Semáforos sin Espera Activa

- Con cada semáforo hay una cola de espera asociada. Con cada semáforo hay asociados dos elementos:
 - un valor (de tipo entero)
 - un puntero al primer proceso de la cola de espera
- Dos operaciones:
 - **block** – coloca el proceso llamante en la cola de espera apropiada
 - **wakeup** – saca un proceso de la cola de espera y lo coloca en la cola de listos

Implementación de Semáforos sin Espera Activa

- Implementación de wait:

```
wait (S){  
    valor--;  
    if (valor < 0) {  
        añade este proceso a la cola de espera  
        block();  
    }  
}
```

- Implementación de signal:

```
signal (S){  
    valor++;  
    if (valor <= 0) {  
        saca un proceso P de la cola de espera  
        wakeup(P);  
    }  
}
```

Bloqueos mutuos e Inanición

- Bloqueos mutuos (deadlock) – dos o más procesos esperan indefinidamente un evento que sólo puede ser causado por uno de los procesos que esperan
- Sean s y q dos semáforos inicializados a 1

P_0	P_1
wait (S);	wait (Q);
wait (Q);	wait (S);
.	.
.	.
.	.
signal (S);	signal (Q);
signal (Q);	signal (S);

- Inanición – bloqueo indefinido. Un proceso puede no ser nunca sacado de la cola de espera de un semáforo

Monitores

- En el estudio y uso de los semáforos se puede ver que las llamadas a las funciones necesarias para utilizarlos quedan repartidas en el código del programa, haciendo difícil corregir errores y asegurar el buen funcionamiento de los algoritmos. Para evitar estos inconvenientes se desarrollaron los monitores.
- Fueron definidos por primera vez por Charles Antony Richard Hoare en un artículo del año 1974.
- Son **estructuras de datos abstractas** destinadas a ser usadas sin peligro por más de un hilo de ejecución, ya que sus métodos son ejecutados con exclusión mutua (un hilo como máximo puede estar ejecutando cualquiera de sus métodos).
- Simplifica el razonamiento de implementar monitores en lugar de código a ser ejecutado en paralelo.

Monitores

Un monitor tiene cuatro componentes: inicialización, datos privados, métodos del monitor y cola de entrada.

- **Inicialización:** contiene el código a ser ejecutado cuando el monitor es creado
- **Datos privados:** contiene los procedimientos privados, que sólo pueden ser usados desde dentro del monitor y no son visibles desde fuera
- **Métodos del monitor:** son los procedimientos que pueden ser llamados desde fuera del monitor.
- **Cola de entrada:** contiene a los hilos que han llamado a algún método del monitor pero no han podido adquirir permiso para ejecutarlos aún.

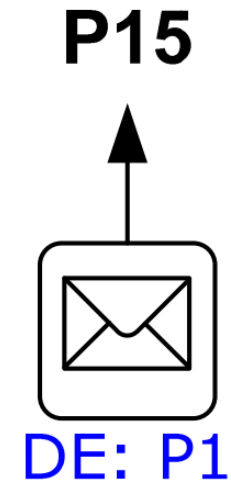
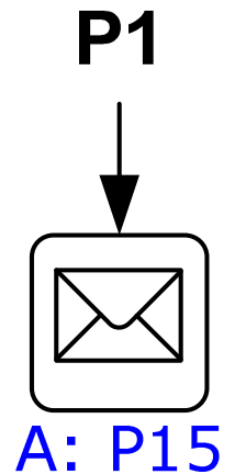
**Se puede ver la implementación en el Problema de los productores y consumidores más adelante*

Comunicación por mensajes

- Los sistemas por mensajes permiten la comunicación entre procesos con espacios de direcciones distintos, sean locales o remotos.
- Operaciones básicas (proporcionadas por el SO):
 - **send(destino,mensaje)**: enviar mensaje a destino,
 - **receive(origen,mensaje)**: recibir mensaje de un destino.
- Tipos de comunicación:
 - Según el modo de nombrar origen y destino
 - Comunicación directa => proceso a proceso
 - Comunicación indirecta => basada en puertos o buzones
 - Según la capacidad del enlace
 - Comunicación síncrona => el enlace tiene capacidad = 0
 - Comunicación asíncrona => el enlace tiene capacidad > 0

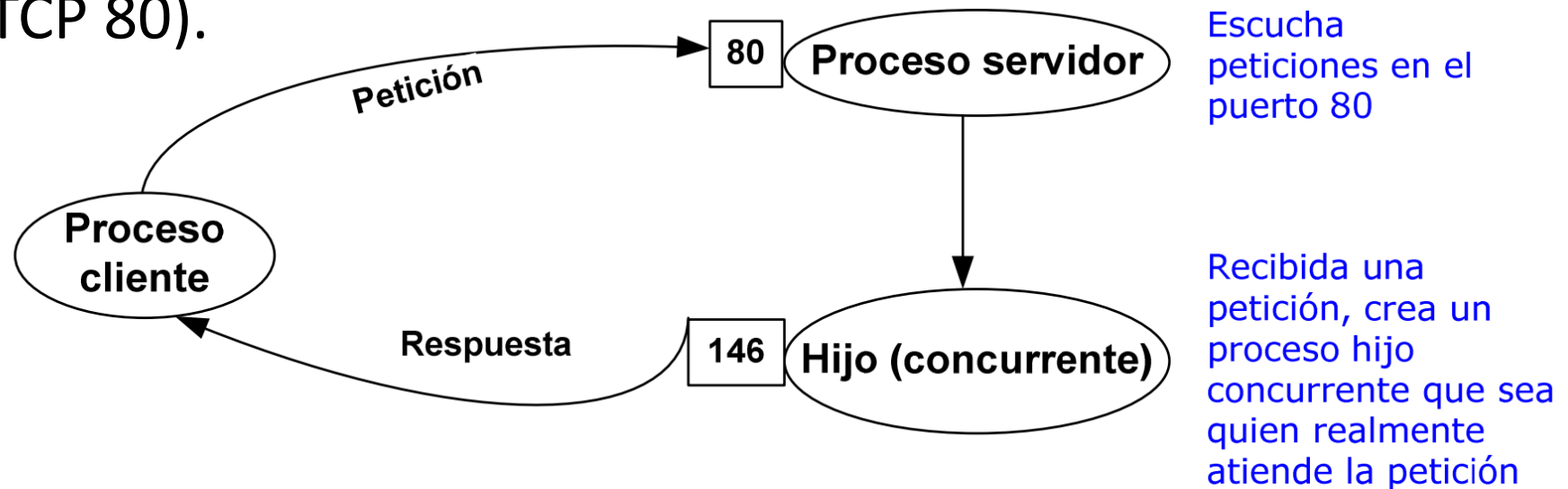
Comunicación por mensajes directa

- Los mensajes se direccionan a procesos
- Los procesos necesitan conocer sus identificadores (destino y origen son identificadores de procesos – PIDs)



Comunicación por mensajes indirecta

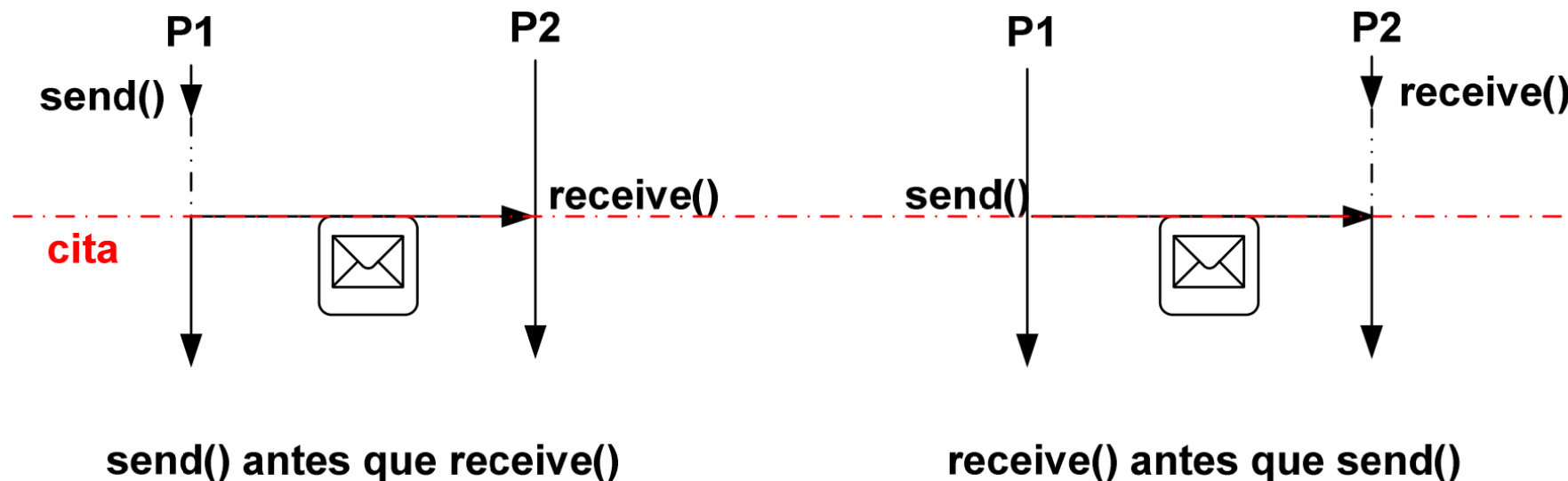
- Los mensajes no se envían directamente a los procesos, sino a puertos: destino y origen son identificadores de puertos.
- Cada puerto tiene un identificador único en el sistema.
- Ejemplo: TCP/IP utiliza puertos bien conocidos. (FTP: TCP 20 y 21, HTTP: TCP 80).



Ventaja: el padre sólo escucha peticiones. Si la generación de una respuesta durase 1 hora, y no se lanzasen hijos para atender las peticiones concurrentemente, no se podría atender más que 1 petición/hora

Comunicación por mensajes síncrona

- ¿Qué significa que la capacidad de un enlace sea cero?
 - El enlace no es capaz de almacenar mensajes: no puede existir ningún mensaje que se haya enviado y esté pendiente de entrega.
- “**Cita**”: se dice que la comunicación sólo tiene lugar cuando (**rendez vous**):
 - El emisor ha invocado *send()*,
 - Y el receptor ha invocado *receive()*.
- ¿Cómo implementar la cita?: El primer proceso que llegue a la cita se suspende hasta que el otro proceso llegue a la cita.



Comunicación por mensajes asíncrona

- La capacidad del enlace es N: puede haber N mensajes enviados y pendientes de ser entregados.
- Sincronización:
 - Receive de enlace que está vacío:
 - Opción 1 (bloqueante) => suspensión al proceso que invoca receive().
 - Opción 2 (no bloqueante) => devolver error al proceso que invoca receive().
 - Send a enlace lleno:
 - Opción 1 (bloqueante) => suspensión al proceso que invoca send().
 - Opción 2 (no bloqueante) => devolver error al proceso que invoca send().

Problemas Clásicos de Sincronización

- Problema de los productores y consumidores (buffer limitado)
- Problema de los lectores y escritores
- Problema de los filósofos
- Problema del barbero dormilón

Problema de los productores y consumidores

- Se describen dos procesos, productor y consumidor, ambos comparten un buffer de tamaño finito. La tarea del productor es generar un producto, almacenarlo y comenzar nuevamente; mientras que el consumidor toma (simultáneamente) productos uno a uno. El problema consiste en que el productor no añada más productos que la capacidad del buffer y que el consumidor no intente tomar un producto si el buffer está vacío.
- Tratemus de dar una solución al problema del productor-consumidor: Usamos una variable entera llamada **count** que guarda el número de elementos en el buffer
 - Inicialmente, itemCount vale 0
 - Es incrementado por el productor cuando produce un nuevo valor y lo almacena en el buffer
 - Es decrementado por el consumidor cuando extrae un elemento del buffer

Problema de los productores y consumidores

```
void producer(){
    while (true){
        item = produceItem();
        if (itemCount == BUFFER_SIZE){
            sleep();
        }
        putIntoBuffer(item);
        itemCount++;
        if(itemCount == 1){
            wakeup(consumer);
        }
    }
}
```

```
void consumidor(){
    while (true){
        if (count == 0){
            sleep();
        }
        item = removeFromBuffer();
        itemCount--;
        if(itemCount == BUFFER_SIZE-1){
            wakeup(producer);
        }
        consumeItem();
    }
}
```

Problema de los productores y consumidores

El problema con este algoritmo es que puede caer en un deadlock, si se considera:

1. El consumidor acaba de consultar la variable itemCount, nota que es cero y pasa a ejecutar el bloque if.
2. Justo antes de llamar a la función sleep() el consumidor es interrumpido y el productor comienza a trabajar.
3. El productor crea un objeto, lo agrega al buffer y aumenta itemCount.
4. Como el buffer estaba vacío antes de la última adición el productor intenta despertar al consumidor.
5. Ahora, el consumidor no estaba durmiendo todavía luego la llamada para despertarlo se pierde. Una vez que el consumidor comienza a trabajar nuevamente pasa a dormir y nunca más será despertado. Esto pasa porque el productor solo lo despierta si el valor de itemCount es 1.
6. El productor seguirá trabajando hasta que el buffer se llene, cuando esto ocurra se pondrá a dormir también.

Como ambos procesos dormirán por siempre, hemos caído en un deadlock.

La esencia del problema es que se perdió una llamada enviada para despertar a un proceso que todavía no estaba dormido. Si no se perdiera, todo funcionaría.

Problema de los productores y consumidores

Solución usando semáforos (resuelve el problema de las llamadas perdidas):

```
semaphore fillCount = 0; // items producidos
semaphore emptyCount = BUFFER_SIZE; // espacio libre

void productor() {
    while (true) {
        item = produceItem();
        down(emptyCount);
        putIntoBuffer(item);
        up(fillCount);
    }
}

void consumidor() {
    while (true) {
        down(fillCount);
        item = removeItemFromBuffer();
        up(emptyCount);
        consumeItem(item);
    }
}
```

Esta solución funciona perfectamente cuando hay solo un productor y un consumidor. Cuando múltiples consumidores o productores comparten el mismo espacio de memoria para almacenar en el buffer, la solución anterior puede llevar a resultados donde dos o más procesos lean o escriban la misma región al mismo tiempo.

Problema de los productores y consumidores

Solución usando monitores (la exclusión mutua está implícita y funciona con cualquier cantidad de productores y consumidores):

```
monitor ProducerConsumer {  
    int itemCount  
    condition full;  
    condition empty;  
  
    void add(item) {  
        while (itemCount == BUFFER_SIZE) {  
            wait(full);  
        }  
        putItemIntoBuffer(item);  
        itemCount = itemCount + 1;  
        if (itemCount == 1) {  
            notify(empty);  
        }  
    }  
}  
  
    void remove() {  
        while (itemCount == 0) {  
            wait(empty);  
        }  
        item = removeItemFromBuffer();  
        itemCount = itemCount - 1;  
        if (itemCount == BUFFER_SIZE - 1) {  
            notify(full);  
        }  
        return item;  
    }  
} //fin monitor
```

Problema de los productores y consumidores

```
void productor() {  
    while (true) {  
        item = produceItem();  
        ProduceConsumer.add(item);  
    }  
}
```

```
void consumidor() {  
    while (true) {  
        item = ProducerConsumer.remove();  
        consumeItem(item);  
    }  
}
```

Problema de los Lectores y Escritores

- Hay un conjunto de datos que se comparte entre varios procesos concurrentes:
 - Lectores – sólo leen el conjunto de datos (no realizan ninguna modificación)
 - Escritores – pueden leer y escribir
- Problema: permitir a muchos lectores leer al mismo tiempo. Sólo un escritor puede acceder a los datos compartidos en un instante dado.
- Datos compartidos por los procesos
 - Conjunto de datos
 - Semáforo **mutex** inicializado a 1
 - Semáforo **wrt** inicializado a 1
 - Entero **readcount** inicializado a 0

Problema de los Lectores y Escritores

- Estructura de un proceso escritor

```
while (true) {  
    wait (wrt) ;  
  
    // se realiza la escritura  
  
    signal (wrt) ;  
}
```

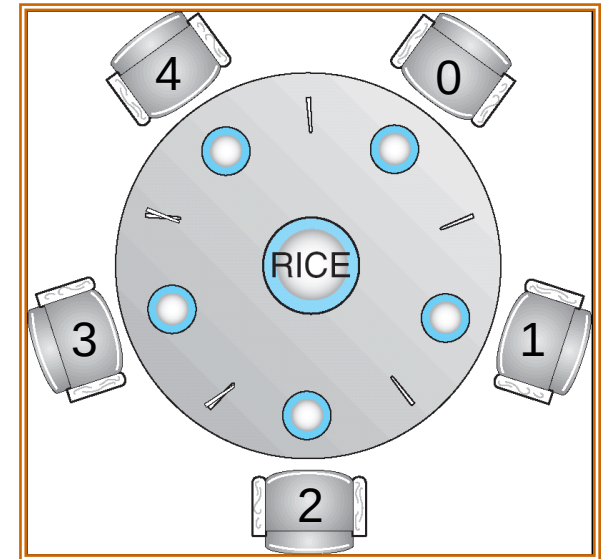
Problema de los Lectores y Escritores

- Estructura de un proceso lector

```
while (true) {  
    wait (mutex) ;  
    readcount ++ ;  
    if (readcount == 1) wait (wrt) ;  
    signal (mutex)  
  
    // se realiza la lectura  
  
    wait (mutex) ;  
    readcount -- ;  
    if (readcount == 0) signal (wrt) ;  
    signal (mutex) ;  
}
```

Problema de los Filósofos

- Es un problema propuesto por Edsger Dijkstra en 1965 para representar el problema de la sincronización de procesos en un sistema operativo.
- **Enunciado:** Cinco filósofos se sientan alrededor de una mesa y pasan su vida cenando y pensando. Cada filósofo tiene un plato de fideos y un tenedor a la izquierda de su plato. Para comer los fideos son necesarios dos tenedores y cada filósofo sólo puede tomar los que están a su izquierda y derecha. Si cualquier filósofo toma un tenedor y el otro está ocupado, se quedará esperando, con el tenedor en la mano, hasta que pueda tomar el otro tenedor, para luego empezar a comer.



Problema de los Filósofos

Análisis del problema:

- Si dos filósofos adyacentes intentan tomar el mismo tenedor a una vez, se produce una condición de carrera: ambos compiten por tomar el mismo tenedor, y uno de ellos se queda sin comer.
- Si todos los filósofos toman el tenedor que está a su derecha al mismo tiempo, entonces todos se quedarán esperando eternamente, porque alguien debe liberar el tenedor que les falta. Nadie lo hará porque todos se encuentran en la misma situación (esperando que alguno deje sus tenedores). Entonces los filósofos se morirán de hambre porque se generó un deadlock.
- El problema consiste en encontrar un algoritmo que permita que los filósofos nunca se mueran de hambre.

Datos compartidos:

- Tazón de arroz (conjunto de datos)
- Semáforos **chopstick [5]** inicializados a 1

Problema de los Filósofos

- Estructura del proceso Filósofo i :

```
while (true) {  
    wait ( chopstick[i] );  
    wait ( chopstick[ (i + 1) % 5] );  
  
    // come  
  
    signal ( chopstick[i] );  
    signal ( chopstick[ (i + 1) % 5] );  
  
    // piensa  
  
}
```

Problema de los Filósofos

La solución anterior es susceptible de sufrir interbloqueo. Algunas soluciones son:

- **Por turno cíclico:** Se empieza por un filósofo, que si quiere puede comer y después pasa su turno al de la derecha. Cada filósofo sólo puede comer en su turno. **Problema:** si el número de filósofos es muy alto, uno puede morir de hambre antes de su turno.
- **Varios turnos:** Se establecen turnos de tiempo fijo. **Problema:** Con base al tiempo que suelen tardar los filósofos en comer y en volver a tener hambre, el tiempo de turno establecido puede hacer que sea peor solución que la anterior.
- **Colas de tenedores:** Cuando un filósofo quiere comer se pone en la cola de los tenedores. Cuando un tenedor está libre lo toma. Cuando toma los dos tenedores, come y deja libre los tenedores. **Problema:** si todos quieren comer a la vez y todos empiezan tomando el tenedor de su derecha se bloquea el sistema; una solución a esto sería que un filósofo que tiene un tenedor espera un tiempo aleatorio para conseguir el segundo tenedor. Si en ese tiempo no queda libre el segundo tenedor, suelta el que tiene y vuelve a ponerse en cola para tener sus dos tenedores.

Problema del barbero dormilón

Enunciado: En una barbería trabaja un barbero que tiene un único sillón de barbero y varias sillas para esperar. Cuando no hay clientes, el barbero se sienta en una silla y se duerme. Cuando llega un nuevo cliente, éste o bien despierta al barbero o —si el barbero está afeitando a otro cliente— se sienta en una silla (o se va si todas las sillas están ocupadas por clientes esperando).

El problema consiste en realizar la actividad del barbero sin que ocurran condiciones de carrera.

Problema del barbero dormilón

```
Semaphore barberoListo = 0;    // (Mutex, sólo 1 o 0)
Semaphore sillasAccesibles = 1; // (Mutex) Si es 1, sillas libres aumenta/disminuye
Semaphore clientes = 0;        // Número de clientes en la sala de espera
int sillasLibres = N;          // N es el número total de sillas
```

```
void barbero(){
    while(true) {
        // Espera la señal de un hilo cliente para despertar
        wait(clientes);
        // (Ya está despierto) Espera señal para poder modificar sillasLibres
        wait(sillasAccesibles);
        sillasLibres ++;
        // El barbero está listo para cortar y manda señal al hilo cliente
        signal(barberoListo);
        // Envía señal para desbloquear el acceso a sillasLibres
        signal(sillasAccesibles);
        // Aquí el barbero corta el pelo de un cliente (zona de código no crítico)
    }
}
```

Problema del barbero dormilón

```
void cliente(){
    wait(sillasAccesibles);    // Espera señal para acceder a sillasLibres
    if (sillasLibres > 0){    // Si hay alguna silla libre, se sienta en una
        sillasLibres--;
        // Envía señal al barbero de que hay un cliente disponible
        signal(clientes);
        // Envía señal para desbloquear el acceso a sillasLibres
        signal(sillasAccesibles);
        // El cliente espera a que el barbero esté listo para atenderlo
        wait(barberoListo);
        // Se le corta el pelo al cliente.
    }
    else{    // Si no hay sillas libres
        // Envía señal para desbloquear el acceso a sillasLibres
        signal(sillasAccesibles);
        // El cliente se va de la barbería (no envía señal de cliente
        disponible)
    }
}
```

Detección y prevención de bloqueos

Detección y prevención de bloqueos

- Interbloqueos
 - Se dan por la necesidad de uso exclusivo de algunos recursos por parte de los procesos
- Ejemplo: proceso A necesita recurso 1 y 2
 proceso B necesita recurso 2 y 1
 - proceso A pide 1
 - proceso B pide 2
 - proceso A pide 2 => se bloquea
 - proceso B pide 1 => se bloquea
 - Interbloqueo
- En dispositivos de E/S, en bases de datos.

Recurso

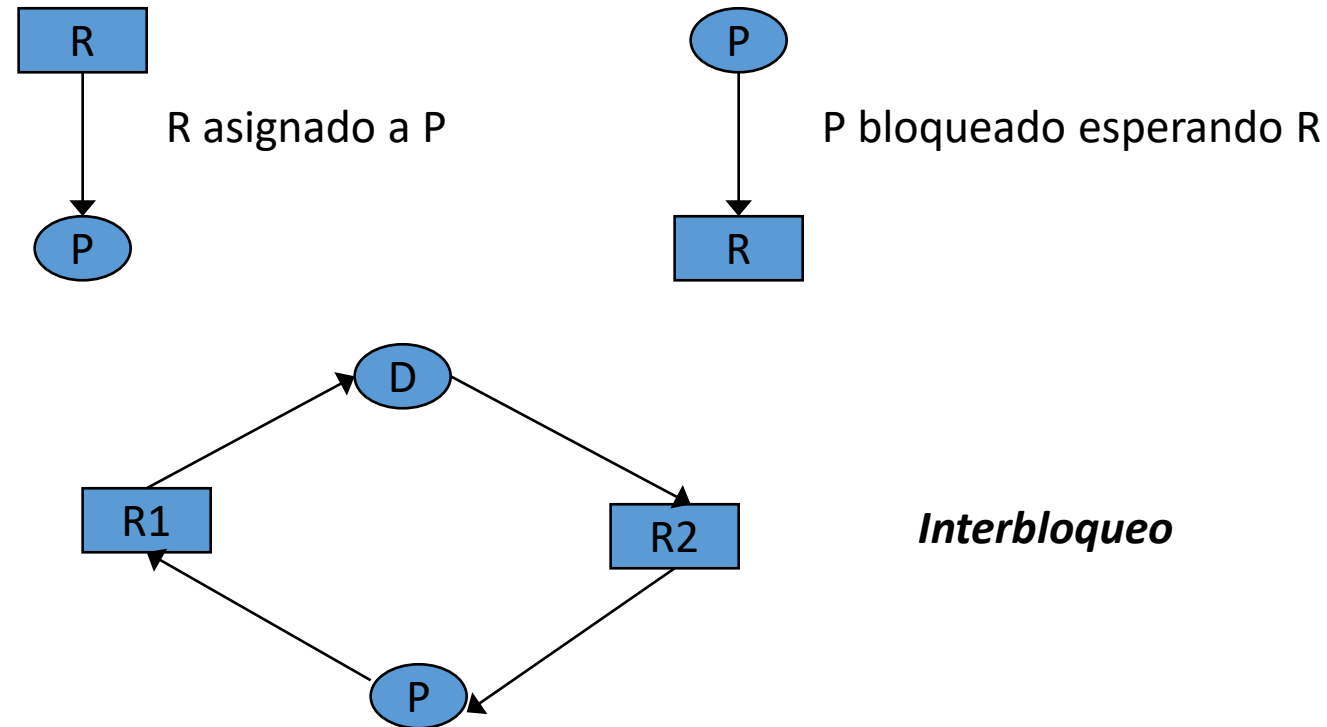
- Sólo puede ser utilizado por un proceso en un instante dado
- Su uso conlleva:
 - Solicitar el recurso
 - Usar el recurso
 - Dejar el recurso
- Si no está disponible, el proceso debe esperar
- Los procesos se interbloquean si cada uno de ellos espera que se genere algún evento que sólo otro proceso del conjunto puede generar.
 - El evento es normalmente la liberación de un recurso que está siendo utilizado por otro proceso.

Condiciones para el interbloqueo

- Enunciadas por Coffman
 - ***Exclusión mutua***: un recurso está asignado a un proceso o está disponible
 - ***Retención y espera***: Se pueden solicitar más recursos sin abandonar los ya obtenidos
 - ***No expulsión***: Los procesos tienen que dejar los recursos explícitamente
 - ***Círculo de espera***: Se produce un círculo de 2 o más procesos

Grafos para modelar interbloqueos

- Los grafos sirven para averiguar si una determinada secuencia produce interbloqueo

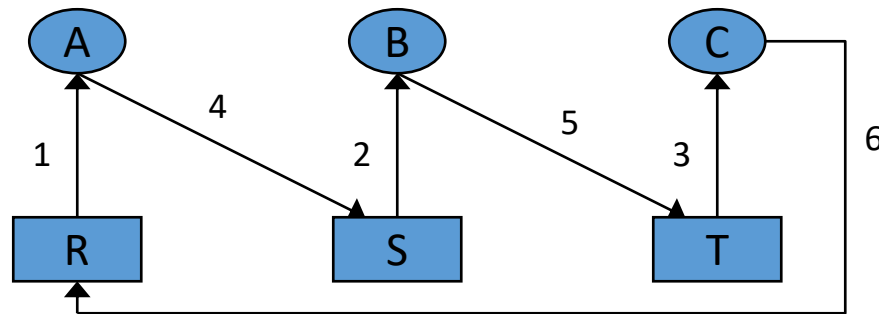


Ejemplo

- 3 procesos

A	B	C
Pedir R	Pedir S	Pedir T
Pedir S	Pedir T	Pedir R
Dejar R	Dejar S	Dejar T
Dejar S	Dejar T	Dejar R

- Planificación por turno rotatorio =====>



- El S.O puede suspender un proceso =>

1. A pide R
2. B pide S
3. C pide T
4. A pide S
5. B pide T
6. C pide R

1. A pide R
2. C pide T
3. A pide S
4. C pide R
5. A deja R
6. A deja S

Formas de tratar el interbloqueo

- El algoritmo del avestruz
 - Esconder la cabeza y desentenderse
 - Unix
- Detección y eliminación de interbloqueos
 - Controlar peticiones y liberaciones de recursos
 - Actualizar el grafo de recursos y comprobar que no haya bucles
 - Comprobar periódicamente si hay procesos que han estado bloqueados durante mucho tiempo
- Prevención de interbloqueos
- Predicción de interbloqueos

Prevención de interbloqueos

- Imponer restricciones a los procesos de forma que un interbloqueo no sea posible
- No habrá interbloqueo si no se cumple alguna de las 4 condiciones necesarias para el interbloqueo:
 - Exclusión mutua
 - No se puede eliminar, se puede llegar a condiciones de carrera
 - Retención y espera
 - Solicitar todos los recursos al principio
 - No permite utilización óptima
 - No se sabe a priori
 - Antes de solicitar un recurso, los procesos están obligados a dejar temporalmente los que tienen retenidos
 - No expulsión
 - No aconsejable
 - Círculos de espera

Círculos de espera

- Retener un recurso en cada momento
 - No es adecuado
- Dar a los recursos una numeración general.
 - Los recursos se solicitan en orden de numeración

A - 1	
B - 2	B y luego D
C - 3	pero no D y luego B
D - 4	
E - 5	

- El grafo no puede tener bucles cerrados
- Puede ser difícil dar a los recursos una numeración adecuada

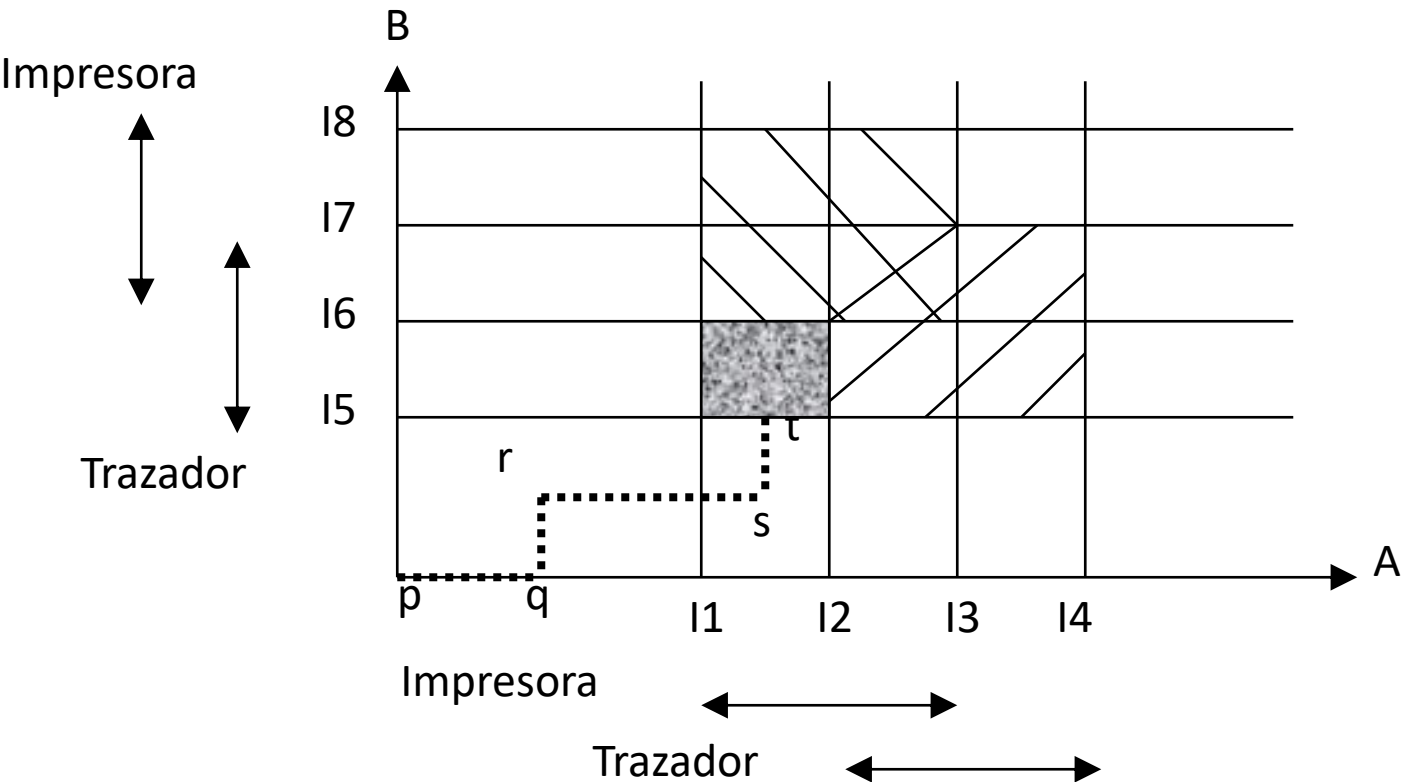
Predicción de interbloqueos

- Evitar los interbloqueos cuando se asignen los recursos
 - Trayectorias de recursos
 - Algoritmo del banquero para un recurso
 - Algoritmo del banquero para múltiples recursos

Trayectorias de recursos

- Para 2 recursos y 2 procesos

..... Ejecución de A
..... Ejecución de B



Algoritmo del banquero para un recurso

- Estado del sistema con respecto a la asignación de recursos: lista de procesos, recursos usados y máximo

	usado	max
p1	0	6
p2	0	5
p3	0	4
p4	0	7

Disponible=10

	usado	max
p1	1	6
p2	1	5
p3	2	4
p4	4	7

Disponible=2

	usado	max
p1	1	6
p2	2	5
p3	2	4
p4	4	7

Disponible=1

- No todos los procesos van a utilizar el máximo
- Estado seguro: si hay una secuencia que lleve a todos los procesos a obtener sus máximos

Algoritmo del banquero para múltiples recursos

- Ra: recursos asignados en ese momento
- Rn: recursos que se necesitan todavía
- E: recursos existentes
- P: recursos asignados a algún proceso
- D: recursos disponibles

Ra

A	3	0	1	1
B	0	1	0	0
C	1	1	1	0
D	1	1	0	1
E	0	0	0	0
	R1	R2	R3	R4

Rn

A	1	1	0	0
B	0	1	1	2
C	3	1	0	0
D	0	0	1	0
E	2	1	1	0
	R1	R2	R3	R4

$E = (6 \ 3 \ 4 \ 2)$

$P = (5 \ 3 \ 2 \ 2)$

$D = (1 \ 0 \ 2 \ 0)$

$P + D = E$

Algoritmo de comprobación de estado seguro

- Buscar fila F cuyas necesidades de recursos sean menores que D. Si no hay=> estado no seguro, el sistema se puede interbloquear, se necesitan más de los disponibles
- Suponer que el proceso de la fila F pide todos los recursos y termina. Marcar el proceso como terminado y añadir sus recursos a D
- Repetir los dos pasos anteriores hasta terminar todos los procesos. Si se puede terminar => estado seguro